Unity (C#) - Complete Source Code

Semester 1 Capstone - Final Worked Example

VIDEO GAME CREATION

The complete, final C# source for the Semester 1 capstone game - Level 1 through Level 3, plus all five of Lesson 14's Level 3 stretch options built in together rather than just one. Assembled from this course's own canonical Teacher's Keys; no new code beyond what those levels and stretch goals call for. This document covers HOW the finished project is built - the reasoning behind each decision lives in the Semester 1 GDD and the lesson materials themselves, not here.

Every file below is the COMPLETE, final version of that script - nothing is partial, and nothing is marked as "new" versus "old," since this is meant to be dropped into a fresh project as-is, not diffed against last week's version.

Required Tags & Layers

- GameObject tags used in code: "Enemy" (on the Enemy prefab) and "Player" (on the Player object) - both referenced by CompareTag() in several scripts below.
- A companion IDE Setup Guide (Inspector fields, prefab wiring, GameObject hierarchy) is a separate document - this one is source code only.

GameManager.cs

Shared game state (Rule 13) - score, lives, wave, win/lose, and now the title-screen gate (gameStarted). Wave 1's enemiesRemaining reset now uses the same 4 + wave formula everywhere (Lesson 14), not a leftover fixed 5. Also restores two Level 3 options the Lesson 14 pass had dropped: Lesson 6's centered OnGUI text (GUIStyle.alignment), and Lesson 10's fade-in instead of an instant pop for the win/lose message. Formatted to match Robert's own working project - aligned declarations, hanging multi-line conditions, closing parens on their own line.

```
using UnityEngine;
using UnityEngine.InputSystem;

public class GameManager : MonoBehaviour
{
    public static int   enemiesRemaining    = 5;
    public static bool  hasWon              = false;
    public static bool  hasLost             = false;
    public static bool  gameActive          = false;
    public static bool  gameStarted         = false;
```

```
    public static int   lives              = 3;
    public static int   score              = 0;
    public static int   wave               = 1;
    public static int   totalWaves         = 3;
    public static bool  nextWaveRequested  = false;
    public static float waveMessageTimeLeft = 0f;
    public static int   startFrame         = -1; // the frame Space started the
game - see Player.cs
    public static float winLoseStartTime   = -1f; // Lesson 10 Level 3: fade the
win/lose text in, not pop it in instantly

    public static GameManager  instance;
    public        EnemySpawner spawner;
    public        AudioClip    lifeLostSound;
    public        AudioClip    gameOverSound;
    public        AudioClip    gameStartSound;
    public        AudioClip    winSound;
    public        AudioClip    waveClip;

    void Awake() { instance = this; }

    void Start()
    {
        wave              = 1;
        enemiesRemaining  = 4 + wave;
        lives             = 3;
        score             = 0;
        totalWaves        = 3;
        winLoseStartTime  = -1f;
        nextWaveRequested = false;
        hasWon            = false;
        hasLost           = false;
        gameActive        = false;
        gameStarted       = false;
    }

    void Update()
    {
        // Level 3 stretch: title screen. Nothing else runs until the player
        // presses Space - spawning is deliberately NOT in EnemySpawner.Start()
        // any more, so the very first wave is only ever spawned from here.
        if (!gameStarted)
        {
            if (Keyboard.current.spaceKey.wasPressedThisFrame)
            {
                gameStarted = true;
                gameActive  = true;
                startFrame  = Time.frameCount; // so Player.cs can tell this Space
press apart from a fire input
                AudioSource.PlayClipAtPoint( instance.gameStartSound
                                           , Camera.main.transform.position
                                           );
                spawner.SpawnEnemies();
            }
            return;
        }

        if (!gameActive && Keyboard.current.rKey.wasPressedThisFrame) { ResetGame();
}

        if (nextWaveRequested)
        {
```

```
            nextWaveRequested   = false;
            spawner.SpawnEnemies();
            waveMessageTimeLeft = 1f;
        }

        if (waveMessageTimeLeft > 0f) { waveMessageTimeLeft -= Time.deltaTime; }
    }

    // Overload kept for any call site that doesn't care about a bonus-enemy
    // point value - defaults to the Lesson 4 baseline of 10 points per kill.
    public static void RegisterKill() { RegisterKill(10); }

    public static void RegisterKill(int points)
    {
        enemiesRemaining--;
        score += points;
        if (enemiesRemaining <= 0) { WaveCleared(); }
    }

    public static void RegisterEscape()
    {
        lives--;
        enemiesRemaining--;
        if (lives <= 0)
        {
            winLoseStartTime = Time.time;
            hasLost          = true;
            gameActive       = false;
            AudioSource.PlayClipAtPoint(instance.gameOverSound,
Camera.main.transform.position);
        }
        else if (enemiesRemaining <= 0)
        {
            WaveCleared();
        }
        else
        {
            AudioSource.PlayClipAtPoint(instance.lifeLostSound,
Camera.main.transform.position);
        }
    }

    static void WaveCleared()
    {
        if (wave >= totalWaves)
        {
            winLoseStartTime = Time.time;
            hasWon           = true;
            gameActive       = false;
            AudioSource.PlayClipAtPoint(instance.winSound,
Camera.main.transform.position);
        }
        else
        {
            wave++;
            enemiesRemaining  = 4 + wave;
            nextWaveRequested = true;
            AudioSource.PlayClipAtPoint(instance.waveClip,
Camera.main.transform.position);
        }
    }
```

```
    public void ResetGame()
    {
        spawner.ClearEnemies();

        // Level 3 stretch: shields are pre-placed, not spawned, so a restart has to
        // put each one back the way Shield.cs found it rather than re-instantiating
it.
        foreach (   Shield shield
                in FindObjectsByType<Shield>(FindObjectsInactive.Include))
                { shield.ResetShield(); }

        score              = 0;
        lives              = 3;
        wave               = 1;
        enemiesRemaining   = 4 + wave;
        waveMessageTimeLeft = 0f;
        winLoseStartTime   = -1f;
        hasWon             = false;
        hasLost            = false;
        nextWaveRequested  = false;
        gameActive         = true;
        startFrame         = Time.frameCount; // R is a fresh start too - keep
Player.cs's guard honest

        spawner.SpawnEnemies();
    }

    // Level 3 stretch: title/game-over screen. Kept as OnGUI (immediate-mode),
    // matching the win/lose banner this course already established in Lesson 10 -
    // not moved to a Canvas, so the one existing message mechanism keeps doing all
of it.
    // Lesson 6 Level 3: centered via GUIStyle.alignment. Lesson 10 Level 3: the
    // win/lose text fades in over half a second instead of just appearing.
    void OnGUI()
    {
        GUIStyle bigStyle        = new GUIStyle(GUI.skin.label);
        bigStyle.fontSize        = 48;
        bigStyle.alignment       = TextAnchor.MiddleCenter;
        bigStyle.normal.textColor = Color.yellow;

        if (!gameStarted)          { GUI.Label(new Rect(10, 10, 1400, 300),
"Kestrel Watch\nPress SPACE to begin your watch", bigStyle); return; }
        if (hasWon || hasLost)
        {
            float fadeInAlpha        = Mathf.Clamp01((Time.time - winLoseStartTime)
/ 0.5f);
            Color c                  = bigStyle.normal.textColor;
            bigStyle.normal.textColor = new Color(c.r, c.g, c.b, fadeInAlpha);
        }
        if (hasWon)                  GUI.Label(new Rect(10, 10, 1400, 300), "You
Win! Press R to Restart"                 , bigStyle);
        if (hasLost)                 GUI.Label(new Rect(10, 10, 1400, 300), "Game
Over - Press R to Restart"               , bigStyle);
        if (waveMessageTimeLeft > 0f) GUI.Label(new Rect(10, 10, 1400, 300), "Wave "
+ wave + "!"                           , bigStyle);
    }
}
```

EnemySpawner.cs

Spawns a wave, sized and speeded by the wave number (Lesson 14). Spawn position uses a starting point that shifts left as the wave's enemy count grows, keeping the row roughly centered rather than drifting - Robert's own working approach, kept as the canonical example. No longer auto-spawns from Start() - GameManager's title-screen gate owns the very first spawn now. Also restores two Level 3 options from earlier lessons that the Lesson 14 pass had dropped: Lesson 9's Random.Range(5, 16) scoring variety, and Lesson 5's alternating striped-row coloring - both applied before the bonus-enemy override below, same as speed already was, so the bonus enemy still reads as clearly better/different. Formatted to match Robert's own working project.

```
using UnityEngine;

public class EnemySpawner : MonoBehaviour
{
    [SerializeField] private GameObject enemyPrefab;

    public void ClearEnemies()
    {
        foreach (  GameObject oldEnemy
                in GameObject.FindGameObjectsWithTag("Enemy")
                )
                { Destroy(oldEnemy); }
    }

    public void SpawnEnemies()
    {
        int count = 4 + GameManager.wave;
        for ( int enemyIndex = 0;
              enemyIndex < count;
              enemyIndex++
            )
        {
            Vector3 pos            = new Vector3((-3f - GameManager.wave) +
enemyIndex * 2f, 4f, 0f);
            GameObject newEnemy    = Instantiate(enemyPrefab, pos,
Quaternion.identity);

            Enemy enemyScript      = newEnemy.GetComponent<Enemy>();
            enemyScript.speed      = 1.5f + (GameManager.wave - 1) * 0.5f;
            enemyScript.pointValue = Random.Range(5, 16);  // Lesson 9 Level 3:
scoring variety

            // Lesson 5 Level 3: alternating striped-row coloring, by spawn slot -
            // deliberately simple shades so it reads as a pattern, not noise.
            newEnemy.GetComponent<SpriteRenderer>().color = ( enemyIndex % 2 == 0)
                                                            ? new Color(0.8f, 0.1f,
0.2f)
                                                            : new Color(0.95f, 0.4f,
0.5f);

            // Level 3 stretch: a bonus enemy - the last slot in every even-numbered
            // wave, worth more points and a little faster, using the exact same
            // Enemy prefab rather than a second one. Overrides both the random
            // point value and the striping above, same as it already overrides
speed.
            if (  enemyIndex           == count - 1
               && GameManager.wave % 2 == 0
               )
```

```
            {
                enemyScript.pointValue = 50;
                enemyScript.speed     += 1f;
                newEnemy.GetComponent<SpriteRenderer>().color = new Color(1f, 0.85f,
0.2f);
            }
        }
        GameManager.enemiesRemaining = count;
    }
}
```

Enemy.cs

The Lesson 13 hit-flash/explosion Coroutine, plus Lesson 14's speed field and the Level 3 bonus-enemy pointValue. Movement is skipped here while FormationController's formation-marching mode is active (see FormationController.cs) - the two movement systems share one Enemy, not two competing Updates. Start() also restores the per-enemy speed variation from an earlier lesson - EnemySpawner.cs sets the wave's base speed on this field BEFORE Instantiate's Start() call ever runs, so multiplying here layers the random variety on top of that base instead of replacing it. Skipped during formation mode: a formation enemy's own speed field is never read while FormationController is moving it, so randomizing it there would do nothing except make the Inspector value misleading.

```
using UnityEngine;
using System.Collections;

public class Enemy : MonoBehaviour
{
    public float          speed      = 1.5f;
    public int            pointValue = 10;
    public GameObject     explosionPrefab;
    public SpriteRenderer spriteRenderer;
    public Sprite         flashSprite;
    public Sprite         normalSprite;

    void Start()
    {
        normalSprite = spriteRenderer.sprite;

        // A little per-enemy variety on top of EnemySpawner's wave-based base
        // speed, so a wave doesn't move as one perfectly uniform wall. Only for
        // lane descent - formation members all share FormationController's one
        // moveSpeed instead, so this field goes unused while marching anyway.
        if (!FormationController.formationModeActive)
            { speed *= Random.Range(0.75f, 1.3f); }
    }

    void Update()
    {
        if (!GameManager.gameActive) return;
        if (FormationController.formationModeActive) return; // formation moves its
own members

        transform.Translate(Vector3.down * speed * Time.deltaTime);
        if (transform.position.y < -6f)
        {
            GameManager.RegisterEscape();
            Destroy(gameObject);
        }
    }
```

```
    // Level 3 stretch: a short flicker (flash/normal/flash) across the same
    0.1 seconds Level 1 already waits, instead of one flat swap.
    public IEnumerator HandleHit()
    {
        Instantiate(explosionPrefab, transform.position, Quaternion.identity);
        spriteRenderer.sprite = flashSprite;
        yield return new WaitForSeconds(0.03f);
        spriteRenderer.sprite = normalSprite;
        yield return new WaitForSeconds(0.03f);
        spriteRenderer.sprite = flashSprite;
        yield return new WaitForSeconds(0.04f);
        Destroy(gameObject);
    }
}
```

Bullet.cs

Hit detection stays here since Lesson 4 - only Bullet ever calls GameManager.RegisterKill(). Now passes the hit enemy's own pointValue through, so a bonus enemy scores correctly without Bullet needing to know what a bonus enemy is. The added other.enabled check is the other half of Shield.cs's same guard - without it, a Bullet and a Shield reaching the same Enemy in one physics step could each register their own kill on it, a double-count that shows up as the wave (and the game) clearing early with an enemy still visibly on screen. Start()/the flashTimer block in Update() restore an earlier lesson's muzzle-flash effect - a brief yellow, slightly-enlarged flash right as the bullet spawns, fading back to normal after 0.1 seconds.

```
using UnityEngine;

public class Bullet : MonoBehaviour
{
    public float     speed = 6f;
    public AudioClip hitSound;
    private float    flashTimer;

    void Start()
    {
        GetComponent<SpriteRenderer>().color = Color.yellow;
        transform.localScale                *= 1.3f;
        flashTimer                           = 0.1f;
    }

    void Update()
    {
        transform.Translate(Vector3.up * speed * Time.deltaTime);
        if (transform.position.y > 6f) Destroy(gameObject);

        if (flashTimer > 0f)
        {
            flashTimer -= Time.deltaTime;
            if (flashTimer <= 0f)
            {
                GetComponent<SpriteRenderer>().color = Color.white;
                transform.localScale /= 1.3f;
            }
        }
    }

    void OnTriggerEnter2D(Collider2D other)
    {
```

```
        if (  other.CompareTag("Enemy")
           && other.enabled)
        {
            other.enabled = false;
            Enemy enemy   = other.GetComponent<Enemy>();
            AudioSource.PlayClipAtPoint(hitSound, Camera.main.transform.position);
            GameManager.RegisterKill(enemy.pointValue);
            enemy.StartCoroutine(enemy.HandleHit());
            Destroy(gameObject);
        }
    }
}
```

Explosion.cs

Fully self-contained - cleans itself up, and (Level 3 stretch) grows slightly over its own lifetime. Knows nothing about score, lives, or waves.

```
using UnityEngine;

public class Explosion : MonoBehaviour
{
    void Start() { Destroy(gameObject, 0.3f); }

    private void Update() { transform.localScale += Vector3.one * Time.deltaTime *
2f; }
}
```

Player.cs

Firing stays a simple cooldown (Lesson 3). Level 3 stretch: ActivateRapidFire() temporarily shrinks the cooldown cap for the power-up pickup (see PowerUp.cs) - the one place that pickup needs to reach into Player at all. Also now checks GameManager.gameActive and GameManager.startFrame, so the exact Space press that starts (or restarts) the game can never also register as a shot - without the frame check, whether that happened would depend on Unity's (unspecified) order between GameManager's and Player's Update(), not on anything in this file.

```
using UnityEngine;
using UnityEngine.InputSystem;
using System.Collections;

public class Player : MonoBehaviour
{
    [SerializeField] private GameObject  bulletPrefab;
    [SerializeField] private float       fireCooldownMax = 0.2f;
    [SerializeField] private float       fireCooldown    = 0f;
    [SerializeField] private float       bulletYOffset   = 0.7f;
    [SerializeField] private AudioSource audioSrc;

    void Update()
    {
        fireCooldown -= Time.deltaTime;
        if (  Keyboard.current.spaceKey.wasPressedThisFrame
           && GameManager.gameActive
           && Time.frameCount != GameManager.startFrame
           && fireCooldown    <= 0f
           )
        {
            Instantiate(bulletPrefab
```

```
                      , transform.position + new Vector3(0, bulletYOffset, 0)
                      , Quaternion.identity
                      );
            fireCooldown = fireCooldownMax;
            audioSrc.Play();
        }
    }

    public void ActivateRapidFire(float duration)
    {
        StartCoroutine(RapidFireRoutine(duration));
    }

    private IEnumerator RapidFireRoutine(float duration)
    {
        float original  = fireCooldownMax;
        fireCooldownMax = 0.05f;
        yield return new WaitForSeconds(duration);
        fireCooldownMax = original;
    }
}
```

PlayerMovement.cs

Unchanged from the Lesson 2-13 reconciliation - a single combined moving check drives the blink, the walking beep, and the four directional sprites together.

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private Sprite         spriteDefault, spriteLeft, spriteRight,
spriteUp, spriteDown;
    [SerializeField] private SpriteRenderer sr;
    [SerializeField] private AudioSource    walkingBeep;
    [SerializeField] private float          xBoundary = 8f;
    [SerializeField] private float          yBoundary = 4.5f;
                     private float          blinkTime = 0f;
                     private float          speed     = 3f;

    void Start() { sr.sprite = spriteDefault; }

    void Update()
    {
        float h = 0f;
        float v = 0f;
        if (Keyboard.current.rightArrowKey.isPressed) h =  1f;
        if (Keyboard.current.leftArrowKey.isPressed)  h = -1f;
        if (Keyboard.current.upArrowKey.isPressed)    v =  1f;
        if (Keyboard.current.downArrowKey.isPressed)  v = -1f;

        transform.Translate(new Vector3(h, v, 0) * speed * Time.deltaTime);
        transform.position = new Vector3( Mathf.Clamp(transform.position.x,
-xBoundary, xBoundary)
                                        , Mathf.Clamp(transform.position.y,
-yBoundary, yBoundary)
                                        , 0
                                        );

        bool moving = h != 0 || v != 0;
```

```
        if (moving)
        {
            blinkTime += Time.deltaTime;
            sr.enabled = ((int)(blinkTime * 6) % 2 == 0);
            if (!walkingBeep.isPlaying) walkingBeep.Play();

            if      (h > 0) sr.sprite = spriteRight;
            else if (h < 0) sr.sprite = spriteLeft;
            else if (v > 0) sr.sprite = spriteUp;
            else            sr.sprite = spriteDown;
        }
        else
        {
            sr.enabled = true;
            blinkTime  = 0f;
            walkingBeep.Stop();
            sr.sprite  = spriteDefault;
        }
    }
}
```

HUD.cs

Score, lives, and wave counter - the permanent on-screen HUD, separate from GameManager's OnGUI win/lose/wave-announcement banner. (The gameActive debug Text field from earlier testing is intentionally not part of this final version - see the Setup Guide's note on that technique.)

```
using UnityEngine;
using UnityEngine.UI;

public class HUD : MonoBehaviour
{
    public Text scoreText;
    public Text livesText;
    public Text waveText;

    void Update()
    {
        scoreText.text = "Score: " + GameManager.score;
        livesText.text = "Lives: " + GameManager.lives;
        waveText.text  = "Wave: " + GameManager.wave + " of " +
GameManager.totalWaves;
    }
}
```

Shield.cs (NEW - Level 3 stretch: destructible shields/barriers)

A few of these sit as pre-placed objects between the enemy wave and the player. Reuses the Lesson 4/13 collision-detection and hit-reaction pattern, just on a new kind of object: an enemy that touches a Shield is destroyed, and the Shield absorbs the hit instead of the player losing a life. Calls GameManager.RegisterKill(0) - zero points, since the player didn't earn this one - so the shield's kill still counts toward enemiesRemaining and the wave-clear/stalemate check same as every other way an enemy can go away. Guards with other.enabled the same way Bullet.cs does, so a Bullet and a Shield hitting the same Enemy in one physics step can't both register a kill on it. Deactivates rather than destroys itself when used up, so GameManager.ResetGame() has something left to find and restore on restart.

```
using UnityEngine;

public class Shield : MonoBehaviour
{
    public int            hitsRemaining = 3;
    public SpriteRenderer spriteRenderer;
    public Color[]        damageColors; // one entry per hit taken, brightest/least-
damaged first
    private int           startingHits;
    private Color         startingColor;

    void Start()
    {
        startingHits = hitsRemaining;
        if (spriteRenderer != null) startingColor = spriteRenderer.color;
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.CompareTag("Enemy")) return;
        if (!other.enabled)             return; // already being handled by a Bullet
(or another Shield) this same frame

        other.enabled = false;
        Destroy(other.gameObject);
        GameManager.RegisterKill(0);           // no points - the shield earned this
kill, not the player
        hitsRemaining--;

        if (  spriteRenderer      != null
           && damageColors        != null
           && damageColors.Length >  0
           )
        {
            int colorIndex       = Mathf.Clamp(damageColors.Length - hitsRemaining,
0, damageColors.Length - 1);
            spriteRenderer.color = damageColors[colorIndex];
        }

        if (hitsRemaining <= 0) gameObject.SetActive(false);
    }

    // Called by GameManager.ResetGame() - shields are pre-placed in the scene,
    // not spawned from a prefab, so restoring one means putting it back the
    // way Start() first found it rather than re-instantiating it.
    public void ResetShield()
    {
        hitsRemaining = startingHits;
        if (spriteRenderer != null) spriteRenderer.color = startingColor;
        gameObject.SetActive(true);
    }
}
```

PowerUp.cs (NEW - Level 3 stretch: a power-up)

```
using UnityEngine;

public class PowerUp : MonoBehaviour
{
    public float fallSpeed = 1f;
```

```
    void Update()
    {
        transform.Translate(Vector3.down * fallSpeed * Time.deltaTime);
        if (transform.position.y < -6f) Destroy(gameObject);
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.CompareTag("Player")) return;

        Player player = other.GetComponent<Player>();
        if (player != null) player.ActivateRapidFire(5f);
        Destroy(gameObject);
    }
}
```

PowerUpSpawner.cs (NEW - Level 3 stretch: a power-up)

Reuses the same cooldown-timer idea as Player.cs's own fire cooldown (Lesson 3), just on a longer, randomized interval.

```
using UnityEngine;

public class PowerUpSpawner : MonoBehaviour
{
    [SerializeField] private GameObject powerUpPrefab;
    [SerializeField] private float      minInterval = 8f;
    [SerializeField] private float      maxInterval = 15f;
                     private float      timeUntilNext;

    void Start() { timeUntilNext = Random.Range(minInterval, maxInterval); }

    void Update()
    {
        if (!GameManager.gameActive) return;

        timeUntilNext -= Time.deltaTime;
        if (timeUntilNext <= 0f)
        {
            Vector3 pos = new Vector3(Random.Range(-3f, 3f), 4f, 0f);
            Instantiate(powerUpPrefab, pos, Quaternion.identity);
            timeUntilNext = Random.Range(minInterval, maxInterval);
        }
    }
}
```

FormationController.cs (NEW - Level 3 stretch: formation-marching movement)

Only one of the two movement systems can own a given Enemy at once, so formationModeActive is computed from the wave number rather than a fixed switch: EVEN waves march in formation, ODD waves use Enemy.cs's own independent lane descent. That gives every run a visible wave-to-wave rhythm - wave 1 (5 enemies, lane descent), wave 2 (6 enemies, formation march), wave 3 (7 enemies, faster lane descent since the per-wave speed formula still applies), and so on - rather than one mode being picked once and staying fixed for the whole run.

```
using UnityEngine;
using System.Collections.Generic;
```

```
public class FormationController : MonoBehaviour
{
    // Even waves march in formation; odd waves use Enemy.cs's own independent
    // lane descent. This is what gives wave 2 a visibly different feel from
    // waves 1 and 3, on top of the count/speed scaling both modes already share.
                    public  static bool  formationModeActive => GameManager.wave %
2 == 0;

    // xBoundary must stay wider than EnemySpawner's own spawn spread (half-width
    // 3 + wave, so 5 on wave 2) or every member starts outside it and the formation
    // free-falls on frame one instead of marching. A small margin (6.5) technically
    // avoids that, but only buys about a second of sideways travel before the first
    // step down, which reads as a fast pass immediately followed by a hard drop -
    // 8 gives a few extra seconds of real marching first.
    [SerializeField] private        float moveSpeed = 1.5f;
    [SerializeField] private        float stepDown  = 0.5f;
    [SerializeField] private        float xBoundary = 8f;
                     private        int   direction = 1;

    void Update()
    {
        if (  !formationModeActive
           || !GameManager.gameActive
           ) return;

        Enemy[] members = FindObjectsByType<Enemy>();
        if (members.Length == 0) return;

        float dx = direction * moveSpeed * Time.deltaTime;
        bool hitEdge = false;
        foreach (Enemy e in members)
        {
            float nextX = e.transform.position.x + dx;
            if (  nextX > xBoundary
               || nextX < -xBoundary
               ) hitEdge = true;
        }

        foreach (Enemy e in members)
        {
            if (hitEdge) e.transform.Translate(Vector3.down * stepDown);
            else         e.transform.Translate(Vector3.right * dx);

            if (e.transform.position.y < -6f)
            {
                GameManager.RegisterEscape();
                Destroy(e.gameObject);
            }
        }

        if (hitEdge) direction *= -1;
    }
}
```